StateScout AI — Track B Execution Handbook

Track: B — Agent & Orchestration (one owner) **Mission:** the brain of StateScout — the LangGraph orchestrator, the BFS exploration loop, the natural-language policy → Expectation Graph pipeline, and system robustness **Toolchain:** Claude Code (terminal agent) as your primary development partner **Capacity:** ~15–20 h/week · 4 months · parent plan: *StateScout Execution Handbook*

Contents

0. What this handbook is

The parent handbook told the team *what* to build and *when*. This one tells **you, the Track B owner**, *how* to build your slice using Claude Code — from a blank machine to the shipped orchestrator. It contains: your exact scope and interfaces (§1), your Claude Code environment set up from scratch (§2), a model-routing policy for every kind of task (§3), the **subagents** and **skills** to create before you start (§4–5), a copy-paste **prompt library** for every milestone task (§6), your month-by-month plan (§7), and working rules that keep an AI-assisted codebase trustworthy (§8).

One honest note up front: Claude Code ships new versions weekly, so specific command flags and defaults drift. Everything here follows the stable primitives (CLAUDE.md, `.claude/agents/`, `.claude/skills/`, hooks, plan mode, `/model`). When a detail matters, `code.claude.com/docs` is authoritative — check it, don't guess.

---

1. Your scope, and the walls around it

1.1 You own

- **The walking-skeleton driver** (Month 1) — the linear script that first wires capture → fingerprint → perceive → check → report end to end.
- **The LangGraph orchestrator** — compiled state machine, Scan→Reason→Act→Observe loop, conditional edges (clean / violated / terminal), checkpointing.
- **The BFS exploration loop** — action enumeration from the AX tree, frontier management, termination, configurable depth limit (FR-10).
- **NL policy → Expectation Graph** (Month 3) — LLM extraction of forbidden/required elements + roles from free-form English, the QA confirmation flow (FR-04/FR-16), serialization of `ExpectationNode`s.
- **Robustness** (Month 4) — error handling, graceful stop, checkpoint-resume, structured logging, per-run manifests.

Your code lives in `apps/agent/orchestrator/` (plus the skeleton driver at `apps/agent/`).

1.2 You do NOT own (and Claude Code must not touch)

Module	Owner	Your relationship to it
`apps/agent/crawler/` (Playwright capture + actions)	Track A	You **call** it. Its capture format is frozen after Month 1.
`apps/agent/perception/` + `apps/agent/negation/`	Track C	You **call** `VLMProvider.analyze()` and the negation engine; never reimplement them.
`apps/agent/graph/` + `services/api/` (Neo4j, fingerprint, dedup, FastAPI)	Track D	You **call** persistence + the visited-set check; D owns the schema your `ExpectationNode`s serialize into.
`apps/vscode-extension/`	Track A	Not your concern beyond the run/stop/status contract.

This matters because Claude Code will happily "fix" a teammate's module while solving your problem — and that creates merge hell and ownership confusion. §2.4 and §4 install guardrails so it can't.

1.3 The interfaces you depend on (freeze in Month 1, week 2)

Your orchestrator is a consumer of three contracts. Get these agreed and written down as typed stubs before you build the real loop, because everything you write composes them:

1. **Capture bundle** (from A): `capture(url_or_action) → {dom, ax_tree, screenshot_path, url}`.
2. **Perception** (from C): `analyze(bundle, role) → SemanticUIMap` and `audit(S_current, C_negative) → list[Violation]`.
3. **Persistence** (from D): `fingerprint(bundle) → str`, `is_visited(state_id, action_id) → bool`, `persist_state/edge/violation(...)`.

Until the real implementations exist, you code against **fakes** you write yourself (§6, prompt M1-P2). That's what lets four people work in parallel.

---

2. Claude Code environment — from scratch

2.1 Install and orient (first session, ~1 hour)

1. Install Claude Code (`npm install -g @anthropic-ai/claude-code`, or the native installer — see `code.claude.com/docs` for your OS) and authenticate with your account.
2. `cd` into the repo, run `claude`, and let your first prompt be `/init` — it scans the repo and drafts a `CLAUDE.md`. **Replace** its draft with the curated version in §2.2; the generated one won't know the track boundaries.
3. Learn the four controls you'll use constantly: **plan mode** (Shift+Tab — Claude proposes a plan before touching files; use it for any multi-file change), `/model` (switch model tier), `/clear` (fresh context between unrelated tasks), and `/agents` (create/manage subagents interactively).

2.2 `CLAUDE.md` — your always-on project memory

This file loads into every session. Keep it short (it costs context every turn) and factual. Create it at the repo root; if the team keeps a shared root `CLAUDE.md`, put the Track-B-specific parts in `apps/agent/orchestrator/CLAUDE.md` instead (directory-level files load when Claude works there).

```
# StateScout AI — agent codebase

## What this is
Autonomous web-UI auditor: LangGraph orchestrator drives a Playwright
crawler, a VLM perceives each UI state, a negation engine flags states
that violate natural-language policies. Graph persisted in Neo4j (CYCLIC
directed graph — never call it a DAG), Redis caches visited pairs.

## Stack & commands
- Python 3.11 via uv. Run: `uv run python -m ...`  Tests: `uv run pytest`
- Lint: `uv run ruff check .`  Types: `uv run mypy apps/agent`
- Services: `docker compose -f infra/docker-compose.yml up -d` (Neo4j :7687, Redis :6379)

## Hard rules (Track B session)
- ONLY create/modify files under: apps/agent/orchestrator/, tests/unit/orchestrator/,
  tests/integration/orchestrator/. Everything else is read-only reference.
- Never modify: apps/agent/crawler/, apps/agent/perception/, apps/agent/negation/,
  apps/agent/graph/, services/api/, apps/vscode-extension/.
- Interfaces to those modules are defined in apps/agent/contracts.py — code against
  them; if a contract seems wrong, STOP and say so instead of changing it.
- TDD: write/adjust the failing test first, then implement, then run pytest.
- Conventional commits with scope, e.g. feat(orchestrator): add BFS frontier.
- Branches: feature/B-<slug> off develop. Never commit to main or develop.
- No new dependencies without asking me first.
- Exploration graph must PRESERVE CYCLES; loop prevention is by visited
  (state_id, action_id) hash set, not by forbidding back-edges.
```

2.3 Permissions

Run `/permissions` once. Sensible baseline for this project: allow file edits and `uv run pytest` / `ruff` without per-call approval; keep `git push`, `docker`, and anything network-touching on ask. Tighten rather than loosen — you review every diff anyway (§8).

2.4 One deterministic guardrail (hook)

CLAUDE.md rules are *suggestions* the model weighs; a **hook** is enforced by the harness and cannot be talked out of. Add a PreToolUse hook that blocks edits outside your track's paths. In `.claude/settings.json`:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          { "type": "command",
            "command": "python3 .claude/hooks/track_b_paths.py" }
        ]
      }
    ]
  }
}
```

`.claude/hooks/track_b_paths.py` reads the tool input JSON from stdin, checks the target path against an allowlist (`apps/agent/orchestrator/`, `tests/**/orchestrator/`, `apps/agent/contracts.py` *read-only*), and exits with code 2 (block, with a stderr message Claude sees) on violation. Ask Claude Code to write this hook as your very first task — it's a perfect warm-up (prompt M0-P1 in §6). Verify the exact hook JSON schema against the hooks page in the docs when you set it up; it's the one part of this setup most likely to drift between versions.

3. Model routing — which model for which task

Claude Code lets you pick the model per session (`/model`) and **per subagent** (the `model:` frontmatter field). The tiers, and the routing policy for Track B:

Tier	Character	Use for (Track B)
Opus (deepest reasoning; `/model opus`)	Slow, expensive, strongest at architecture and subtle invariants	Designing the LangGraph state schema and conditional-edge topology; reasoning about **loop termination** and frontier invariants; designing the policy-extraction prompt (the meta-prompt); reviewing the port from plain-Python loop → LangGraph; any bug where state is corrupted across checkpoints
Sonnet (the default workhorse)	Fast, strong coder, right for 80% of work	All routine implementation: writing nodes, wiring edges, tests, fakes, refactors, logging, config plumbing, error handling
Haiku (fast + cheap)	High-volume, low-stakes	Summarizing long pytest/log output, generating test fixtures and synthetic policy strings, docstrings, changelog drafts — mostly via subagents (§4), not your main session

Rules of thumb: **default to Sonnet**; escalate to Opus only when the task is *design* or *invariant reasoning*, and drop back after. Don't run your main session on Haiku — the savings aren't worth weaker code. When a model choice matters for a specific prompt in §6, it's marked. (Model lineups change; `/model` shows what's current. As of mid-2026 the API tiers are Opus 4.8 / Sonnet 4.x / Haiku 4.5.)

Effort/thinking: for the two genuinely hard reasoning moments (termination proof, checkpoint-resume semantics), ask for extended thinking explicitly in the prompt ("think hard about the invariants before proposing code") and use plan mode so the reasoning lands as a reviewable plan, not a surprise diff.

4. Subagents to create before you start

Subagents are Markdown files in `.claude/agents/`, each with YAML frontmatter (`name` , `description` , optional `model` , `tools`) and a body that becomes that agent's system prompt. They run in **isolated context** — heavy exploration or test noise happens there and only a summary returns to your session. Create these four via `/agents` (paste the bodies below). They're version-controlled — commit them so future-you and teammates get them free.

4.1 `langgraph-scout` — docs & pattern researcher

Why: LangGraph's API is the thing you know least; letting research chew through your main context is how sessions die.

```
---
name: langgraph-scout
description: Researches LangGraph APIs, patterns, and best practices.
  Use when the user needs to know how StateGraph, conditional edges,
  checkpointers, interrupts, or Send/Command APIs work, or wants
  current idiomatic patterns before designing orchestrator code.
model: sonnet
tools: Read, Grep, Glob, WebSearch, WebFetch
---
You are a LangGraph research specialist for the StateScout project.
Given a question, consult the official LangGraph docs
(langchain-ai.github.io/langgraph) and our vendored examples first.
Return: (1) a direct answer, (2) a minimal runnable code sketch,
(3) version caveats, (4) doc links. Never edit files. Prefer the
current stable API; flag anything deprecated. Be concise — the main
agent will implement; you only inform.
```

4.2 `test-runner` — verification worker

Why: pytest output is long and noisy; run it out-of-context and get back a verdict.

```
---
name: test-runner
description: Runs the orchestrator test suite and reports results.
  Use after any implementation change, before any commit, or when
  the user asks "do the tests pass?"
model: haiku
tools: Bash, Read, Grep
---
Run: uv run pytest tests/unit/orchestrator tests/integration/orchestrator -q
Then uv run ruff check apps/agent/orchestrator and
uv run mypy apps/agent/orchestrator.
Report: pass/fail counts, then for each failure the test name, the
assertion, and the 5 most relevant traceback lines. Suggest the likely
faulty file:line. Never modify files.
```

4.3 `fresh-eyes-reviewer` — independent pre-PR review

Why: the agent that wrote the code is the worst judge of it. Isolation gives you genuinely fresh eyes; Opus makes them sharp. Invoke before every PR.

```
---
name: fresh-eyes-reviewer
description: Independent code review of orchestrator changes before a
  PR. Use when the user says "review this branch/diff" or before
  merging to develop.
model: opus
tools: Read, Grep, Glob, Bash
---
You are reviewing a diff you did not write. Run git diff develop...HEAD
and review ONLY what changed. Check, in order:
1. Boundary violations — any file touched outside apps/agent/orchestrator/
   and its tests is an automatic FAIL.
2. Contract fidelity — calls into crawler/perception/graph match
   apps/agent/contracts.py exactly.
3. Termination — could this change make exploration loop forever or
   re-visit a (state, action) pair? Trace the frontier logic.
4. Cycle preservation — nothing may prune back-edges from the graph.
5. State-machine integrity — checkpoint/resume still consistent?
6. Tests — do new behaviors have failing-first tests? Are they honest?
Output: verdict (APPROVE / REQUEST CHANGES), findings ranked by
severity with file:line, and the single riskiest thing about this diff.
```

4.4 `fixture-forge` — cheap test-data generator

```
---
name: fixture-forge
description: Generates synthetic test fixtures — fake capture bundles,
  AX-tree JSON, natural-language policy strings with expected parses.
  Use when tests need varied realistic data.
model: haiku
tools: Read, Write, Glob
---
Generate fixtures ONLY under tests/fixtures/orchestrator/. Follow the
shapes in apps/agent/contracts.py exactly. For policy strings, produce
paired files: the English input and the expected structured parse
(forbidden[], required[], role). Include edge cases: ambiguous wording,
double negation ("should never be unable to see"), multiple roles,
contradictory rules. Keep files small and deterministic (seeded).
```

---

5. Skills to create before you start

Skills live at `.claude/skills/<name>/SKILL.md`. Unlike CLAUDE.md they load **on demand** — the description is the trigger Claude matches against your prompt — so they're the right home for procedures too long to keep always-on. Commit these two.

5.1 `orchestrator-conventions` — how we build LangGraph code here

```
---
name: orchestrator-conventions
description: StateScout orchestrator design conventions. Use whenever
  writing or modifying LangGraph nodes, edges, state schemas, or the
  exploration loop in apps/agent/orchestrator/.
---
# Orchestrator conventions
- One TypedDict state schema, defined once in orchestrator/state.py.
  Nodes take state, return partial updates. No hidden globals.
- Node functions are pure w.r.t. I/O: all crawler/VLM/graph calls go
  through the injected contracts object (orchestrator/deps.py), never
  imported directly. This is what makes nodes testable with fakes.
- Conditional edges return one of the literals: "clean", "violated",
  "terminal". Map them explicitly in build_graph().
- The frontier is a deque of (state_id, action) pairs. An action is
  enqueued ONLY after the visited-set check. Every dequeue must either
  execute or log why it was skipped — silent drops are bugs.
- Cycles are preserved: edges are always persisted; only NODE creation
  is deduplicated by fingerprint.
- Checkpoint after every completed iteration via the LangGraph
  checkpointer; the run must be resumable from the last checkpoint.
- Every run gets a run_id; every log line is structured JSON with
  run_id, node, state_id.
- Config (depth limit, rate caps) comes from orchestrator/config.py
  (pydantic settings), never hardcoded.
```

5.2 `pr-prep` — the mechanical merge checklist

```
---
name: pr-prep
description: Prepare a Track B branch for PR to develop. Use when the
  user says "prep this for PR" or "ready to merge".
---
# PR preparation
1. Invoke the test-runner subagent; all green or stop.
2. Invoke the fresh-eyes-reviewer subagent; resolve REQUEST CHANGES.
3. git diff develop...HEAD --stat — confirm every file is inside
   Track B paths; if not, STOP and tell the user.
4. Rebase on latest develop; rerun tests if the rebase touched code.
5. Squash fixups; conventional-commit messages with (orchestrator) scope.
6. Draft the PR description: what/why, how tested, contract changes
   (should be none), risks. Print it for the user to post.
```

Why skills and not more CLAUDE.md? Cost and precision: CLAUDE.md is paid every turn; these load only when triggered, and the conventions skill in particular is long enough that keeping it always-on would crowd out working context.

---

6. The prompt library

Copy-paste starting prompts for each milestone task. Conventions: 🅿 = start in **plan mode** (approve the plan before any file is touched); model noted when it isn't Sonnet. Adjust file paths to reality; these assume the monorepo layout from the parent handbook. The wording matters less than the *structure*: state the goal, the constraints, the definition of done, and what Claude must NOT do.

Month 0/1 — setup and skeleton

M0-P1 · The path-guard hook (Sonnet)

> Create `.claude/hooks/track_b_paths.py`, a Claude Code PreToolUse hook. It reads the tool-call JSON from stdin, extracts the file path for Edit/Write/MultiEdit tools, and exits 0 if the path is under `apps/agent/orchestrator/`, `tests/unit/orchestrator/`, `tests/integration/orchestrator/`, or `tests/fixtures/orchestrator/`; otherwise print a one-line reason to stderr and exit 2. Also update `.claude/settings.json` to register it (this settings edit is the one allowed exception). Check the current hooks docs for the exact stdin schema before writing. Add a tiny README in `.claude/hooks/`.

M1-P1 · Contracts file first 🅿 (Sonnet)

> Read the parent handbook section on Track interfaces (I'll paste it) and create `apps/agent/contracts.py`: typed Protocol classes + dataclasses for CaptureBundle, SemanticUIMap, Violation, and the three interfaces CrawlerPort, PerceptionPort, GraphPort exactly as described. Pure types, zero logic, zero external imports beyond stdlib/typing/pydantic. Then write `tests/unit/orchestrator/test_contracts.py` that just constructs each type. This file will be frozen after team review — flag anything ambiguous instead of inventing semantics.

M1-P2 · Fakes for parallel work (Sonnet)

> Create `apps/agent/orchestrator/fakes.py`: in-memory fake implementations of CrawlerPort, PerceptionPort, GraphPort from contracts.py. The fake crawler serves a tiny scripted "web app" defined as a dict of states→actions→next-states (include at least one cycle: login ↔ register). The fake perception returns canned SemanticUIMaps and flags a violation when a state contains element "admin-link" and role is "guest". The fake graph tracks visited pairs in a set. Deterministic, no network, no docker. Tests proving each fake honors its Protocol.

M1-P3 · Walking-skeleton driver 🅿 (Sonnet)

> Build `apps/agent/skeleton.py`: a LINEAR script, no LangGraph, that wires the ports end to end for ONE state: capture → fingerprint → persist node → analyze → check one hardcoded negation rule → print a ViolationRecord. It must run in two modes: `--fake` (uses fakes.py, no services) and `--live` (real ports, once teammates land them). Argparse, structured JSON logging with a run_id, exit code 1 if a violation is found (useful in CI). Test with the fakes. Do not implement any real port logic — that's other tracks.

M1-P4 · LangGraph PoC stub 🅿 (**Opus for the plan**, Sonnet to implement)

> First use the langgraph-scout subagent to confirm the current idiomatic APIs for StateGraph, conditional edges, and checkpointers. Then, in plan mode, design a minimal compiled graph in `apps/agent/orchestrator/poc.py` with stub nodes Scan→Reason→Act→Observe and a conditional edge from Reason with branches clean/violated/terminal (terminal just ends). State schema per the orchestrator-conventions skill. In-memory checkpointer. A test that runs the compiled graph over the fake app for 3 iterations and asserts the visited sequence. This is a scaffold for Month 2 — keep it under ~150 lines.

Month 2 — autonomy

M2-P1 · BFS loop, plain Python first 🅿 (Sonnet; **Opus review after**)

> Implement `apps/agent/orchestrator/explore.py`: a plain-Python BFS exploration loop over the ports (no LangGraph yet). Enumerate candidate actions from the capture bundle's AX tree via CrawlerPort; before enqueueing, check GraphPort.is_visited(state_id, action_id); persist every edge (cycles preserved), dedupe only node creation by fingerprint; respect config.depth_limit (FR-10). Definition of done: on the fake app with cycles, the loop (a) terminates, (b) visits every reachable state exactly once per unique (state,action), (c) records the back-edge of the login↔register cycle. Write those three as tests FIRST, watch them fail, then implement. Think hard about the termination invariant and state it in a docstring.

Then, fresh session, `/model opus`: "*Review explore.py solely for termination and coverage invariants. Construct an adversarial fake app (self-loops, diamond, two interleaved cycles, an action that mutates state non-deterministically) and reason about whether the loop can hang or skip states. Propose test cases, not fixes.*"

M2-P2 · Port the loop into LangGraph 🅿 (**Opus plan**, Sonnet implement)

> We're porting explore.py into the compiled LangGraph from poc.py. In plan mode, propose the mapping: which parts of the BFS loop become nodes vs. state vs. conditional edges; where the checkpointer saves; how depth-limit and frontier-exhaustion each route to terminal. Constraint: behavior must be test-identical — the M2-P1 test suite must pass unchanged against the LangGraph version behind the same run() signature. After I approve the plan, implement in `orchestrator/graph_runner.py`, keep explore.py as the reference implementation until parity is proven, then mark it deprecated.

M2-P3 · Config & rate coordination (Sonnet)

> Add `orchestrator/config.py` (pydantic-settings): depth_limit, max_states, perception_rate_per_min, checkpoint_dir, run_id strategy. The orchestrator must await a rate-limiter token before each PerceptionPort call so Track C's Gemini throttle is respected from our side too. Env-overridable, documented in the module docstring, tests for precedence.

Month 3 — policy pipeline

M3-P1 · Design the extraction prompt itself 🅿 (**Opus**)

> Task: design (not yet code) the LLM prompt that turns a QA engineer's free-form English policy into our structured parse: {forbidden: [...], required: [...], role: str, notes}. Use the paired fixtures from fixture-forge as the eval set. The CVPR 2025 negation paper says models mishandle negation — so the prompt must force explicit polarity handling: have the model restate each clause as MUST-NOT-EXIST / MUST-EXIST before emitting JSON, and emit an "ambiguities" list instead of guessing. Deliverable: the prompt text, the JSON schema, and a scoring script spec that measures exact-match on the fixture pairs. We iterate here until ≥90% on fixtures before writing any pipeline code.

M3-P2 · The parser pipeline + confirmation flow 🅿 (Sonnet)

> Implement `orchestrator/policy.py` : PolicyParser takes English text, calls the LLM with the approved M3-P1 prompt (through PerceptionPort's text-LLM method — do not create a second LLM client), validates the JSON against the schema, and returns InterpretedPolicy or raises PolicyAmbiguous with the ambiguity list. Then implement the FR-04/FR-16 confirmation flow as a state: the orchestrator must not start crawling until a confirm() is received (exposed via the run-control API contract to Track D; for now, a callback in deps). Serialize confirmed constraints as ExpectationNodes via GraphPort using Track D's schema — read their schema module, don't redefine it. Tests: happy path, ambiguous policy, correction round-trip.

Month 4 — robustness

M4-P1 · Graceful stop + checkpoint-resume 🅿 (**Opus plan**, Sonnet implement)

> Plan mode. Two features with entangled semantics: (1) graceful stop — a stop signal (from Track D's /stop endpoint contract) must finish the in-flight iteration, checkpoint, persist a run manifest, and exit cleanly; (2) resume — starting with a run_id resumes from the last checkpoint with the frontier and visited-set intact, and must not re-execute the last completed action. Think hard about the crash-DURING-checkpoint case and define which of at-most-once / at-least-once we guarantee per side effect (Neo4j writes are idempotent by fingerprint — lean on that). Plan first with the invariants written out; then tests (kill the run mid-iteration in a test harness), then implementation.

M4-P2 · Structured logging + run manifest (Sonnet)

> Add orchestrator-wide structured logging (JSON lines: ts, run_id, node, state_id, event, duration_ms) and a per-run manifest written at start and finalized at end: config snapshot, git SHA, counts (states, edges, violations, skips), termination reason. No print(). Redact anything resembling credentials (NFR-11). Tests assert manifest completeness on normal end, stop, and resume.

M4-P3 · Edge-case sweep (Sonnet, with **fixture-forge** + **test-runner**)

> Using fixture-forge, generate adversarial fake apps: state explosion (fan-out 50), infinite-scroll-like self-loops, actions that error, perception timeouts, a policy matching zero states. For each, write a test asserting defined behavior (bounded by config, logged skip, retry-then-mark-failed, clean empty report). Fix the orchestrator where behavior is undefined. Run everything through test-runner and give me the final pass matrix.

Everyday micro-prompts

- "*Run the test-runner subagent and summarize.*" — after every change.
- "*Use pr-prep.*" — before every merge.
- "*Ask langgraph-scout whether \<API question> — don't guess from memory.*" — whenever LangGraph syntax is uncertain.
- "*Explain the diff you just made in three bullets and list anything you're unsure about.*" — cheap honesty check before you review.

---

7. Month-by-month plan (Track B view)

Month	Deliverables	Done means
1	Path-guard hook · `contracts.py` (frozen wk 2) · `fakes.py` · walking skeleton (`--fake` + `--live`) · LangGraph PoC	Skeleton detects the planted violation on a broken test-app in `--live` at the team demo; PoC graph runs 3 iterations in CI
2	Plain BFS loop · adversarial termination review · LangGraph port at test-parity · config + rate limiter	Autonomous crawl of the multi-page test-app terminates; cyclic graph visible in Neo4j; 0% duplicate (state,action) — NFR-05
3	Extraction prompt ≥90% on fixtures · `policy.py` + PolicyAmbiguous · FR-04/16 confirmation gate · ExpectationNode serialization	From the extension (Track A), an English policy round-trips: parse → confirm → crawl → violations reference the right policy rule
4	Graceful stop · checkpoint-resume · structured logs + manifest · edge-case sweep green	Kill-and-resume test passes; stop from the API is clean; production checklist items owned by B all ticked; `v1.0`

Budget reality: at 15–20 h/week you have roughly 60–75 hours per month. The `P` plan-mode prompts front-load design so implementation hours aren't wasted on rework — do not skip them to "save time"; that trade always loses on this kind of state-machine code.

---

8. Working rules for an AI-assisted codebase

1. **You are the engineer of record.** Claude Code writes fast; you own correctness. Read every diff before it lands. Anything you can't explain at review, you don't merge.
2. **One task, one session.** `/clear` between unrelated tasks. Long mixed sessions degrade — the model starts deprioritizing your early constraints.
3. **Plan mode for anything multi-file.** Approving a plan is minutes; unwinding a wrong 15-file diff is hours.
4. **Tests are the contract with yourself.** Failing test first (the conventions skill enforces the habit); the test-runner subagent after every change; never merge red.
5. **Fresh eyes before every PR.** The `fresh-eyes-reviewer` exists because the author-model is systematically blind to its own bugs. Isolation + Opus is your second engineer.
6. **Guardrails are layered.** CLAUDE.md persuades, the hook enforces, the reviewer catches. If Claude ever proposes editing another track's module, the answer is no — raise the contract issue with the owner instead.
7. **Don't let Claude invent LangGraph APIs.** It's a fast-moving library; hallucinated method names compile in your head and fail at runtime. `langgraph-scout` first, code second.
8. **Commit small, merge often.** Feature branches `feature/B-<slug>`, PRs to `develop` every few days per the team GitFlow. Small diffs are also the diffs Claude reviews best.
9. **Watch your context.** If a session gets sluggish or forgetful, checkpoint your progress in a `NOTES.md`, `/clear`, and resume with a two-line recap. Cheaper than fighting a degraded session.
10. **Log what the AI got wrong.** Keep a short `docs/ai-postmortems.md`; patterns you record become new CLAUDE.md rules or skill lines. This is how the setup compounds.

---

9. First-week checklist

- [ ] Claude Code installed, authenticated, `/init` run, curated `CLAUDE.md` committed
- [ ] `/permissions` configured (edits + pytest allowed; push/docker on ask)
- [ ] Path-guard hook written (M0-P1) and verified — try to edit `apps/agent/crawler/x.py` and watch it block
- [ ] Four subagents committed under `.claude/agents/` (§4)
- [ ] Two skills committed under `.claude/skills/` (§5)
- [ ] `contracts.py` drafted (M1-P1) and circulated to Tracks A/C/D for the freeze
- [ ] `fakes.py` + first green test in CI
- [ ] Walking skeleton runs in `--fake` mode end to end

This handbook is Track B's operational reference. Where it conflicts with the parent StateScout Execution Handbook on scope or schedule, the parent wins; where Claude Code's current docs conflict with a mechanical detail here, the docs win.